

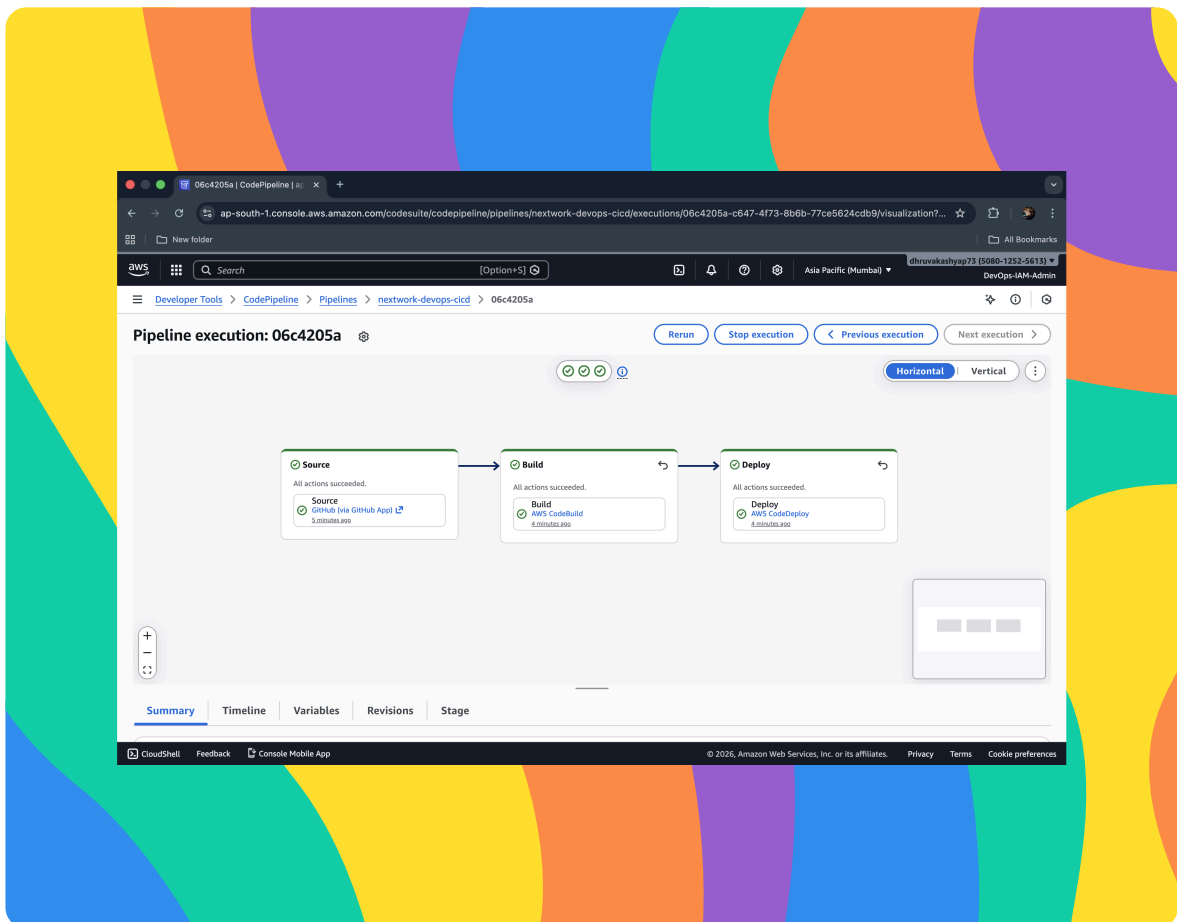


[nextwork.org](https://nextwork.org)

# Build a CI/CD Pipeline with AWS



Dhruva Kashyap





# Introducing Today's Project!

Today we are building an automated CI/CD pipeline using AWS CodePipeline to connect GitHub, CodeBuild, and CodeDeploy into one workflow. This is done so code changes automatically trigger a new build, create updated artifacts, and deploy them without any manual steps. It makes the entire deployment process consistent, faster, and easier to monitor and recover if something fails.

## Key tools and concepts

In this project, we learned key AWS services like EC2, CodeArtifact, CodeBuild, CodeDeploy, CloudFormation, and CodePipeline to build a complete CI/CD workflow. We also understood core DevOps concepts such as version control with Git/GitHub, artifact packaging, automated testing, Infrastructure as Code (IaC), webhooks, and deployment rollback. Together, these tools and concepts helped automate code changes from source to production with reliable and repeatable deployments.

## Project reflection

It took around 120 minutes to complete this project.



## Starting a CI/CD Pipeline

AWS CodePipeline is a fully managed service that automates the process of moving code changes through build and deployment stages. It is used to automatically trigger CodeBuild for compiling and testing code and CodeDeploy for deploying the latest build whenever changes are pushed to GitHub. This creates a reliable and consistent CI/CD workflow with minimal manual effort and reduced human error.

CodePipeline has three execution modes: Superseded, Queued, and Parallel. Superseded cancels the older execution when a new one starts so only the latest code is processed, Queued runs executions one after another in order, and Parallel allows multiple executions to run at the same time independently. Choosing the right mode helps control how your pipeline behaves when many code changes happen quickly.

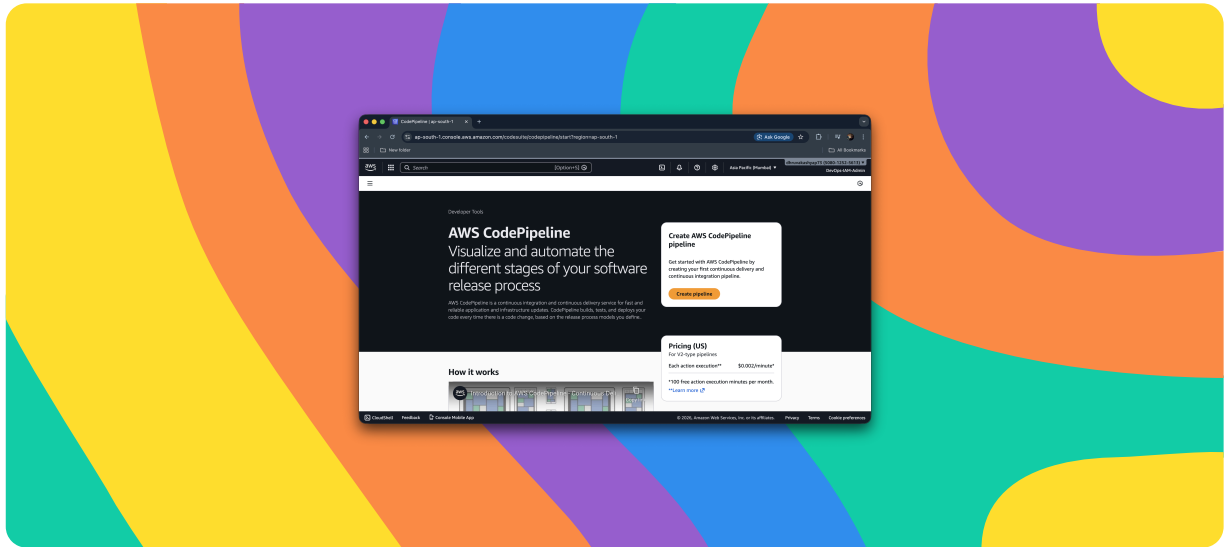
CodePipeline has a service role so it can securely access and manage the AWS services needed to run the pipeline on your behalf. This role gives CodePipeline permissions to interact with resources like S3 for artifacts, CodeBuild for builds, and CodeDeploy for deployments. It ensures everything runs automatically while following least privilege access instead of using your personal credentials.



**Dhruva Kashyap**

NextWork Student

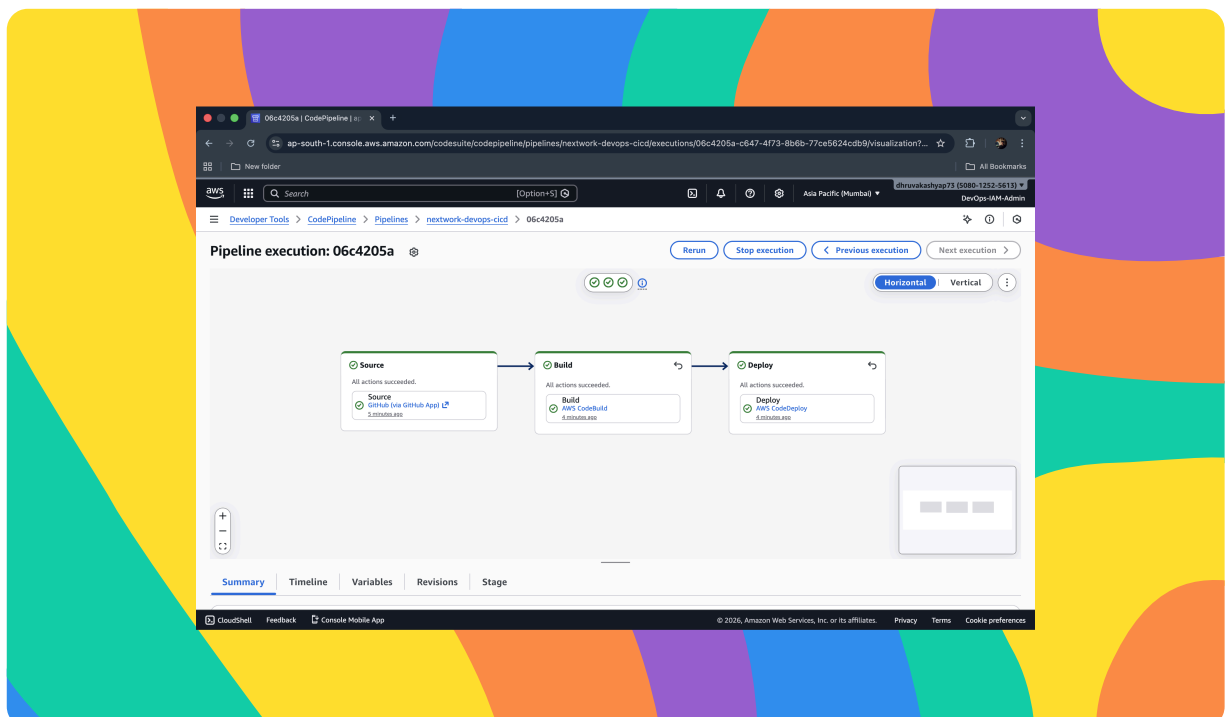
[nextwork.org](https://nextwork.org)



## CI/CD Stages

The three stages in the CI/CD pipeline are Source, Build, and Deploy. The Source stage pulls the latest code from GitHub, the Build stage uses CodeBuild to compile and package the application into an artifact, and the Deploy stage uses CodeDeploy to deploy that artifact to the EC2 instance. These stages work together to automate the full workflow from code changes to deployment.

The pipeline shows the status of each stage (not started, in progress, success, or failed) so you can track progress from Source to Deploy. It also provides execution details like a Stage ID, timestamps, and logs that explain what happened during that stage run. This visibility helps you quickly confirm successful deployments and troubleshoot failures when they occur.

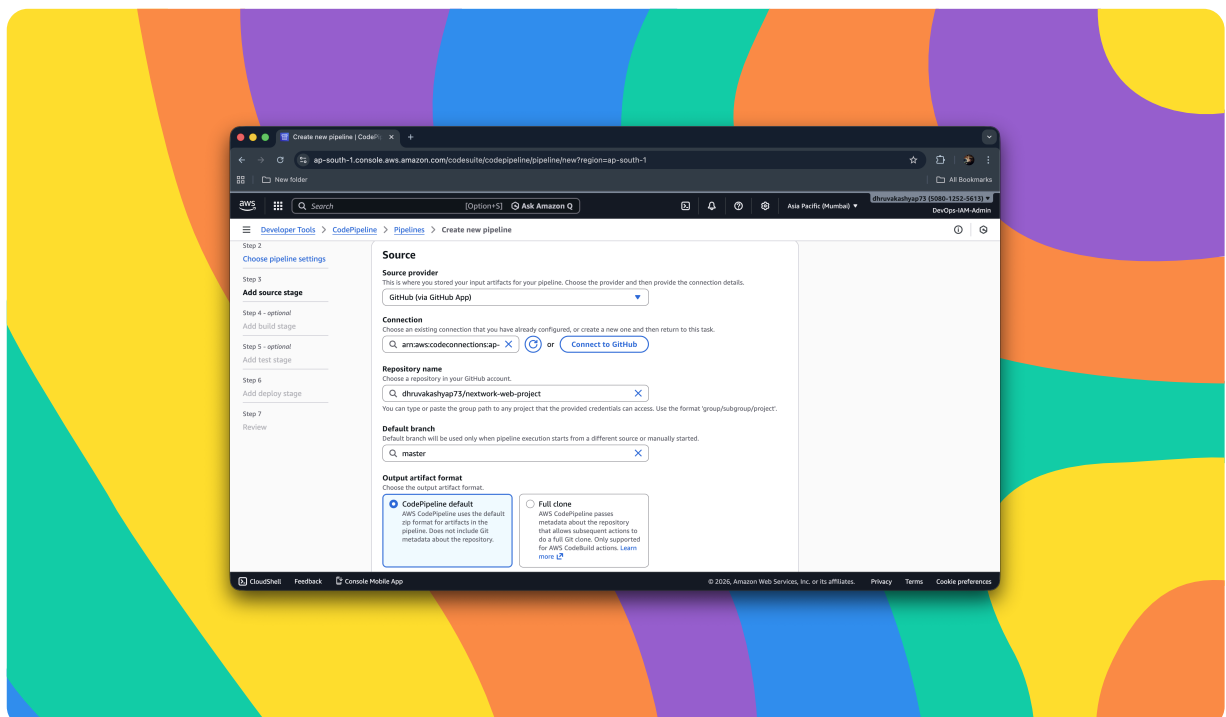




## Source Stage

Specifying a default branch in the Source stage tells CodePipeline exactly which branch to monitor and pull code from when starting a pipeline run. This ensures the pipeline always builds and deploys the correct version of the application code, like the stable master branch. It also enables automatic triggering whenever new commits are pushed to that branch.

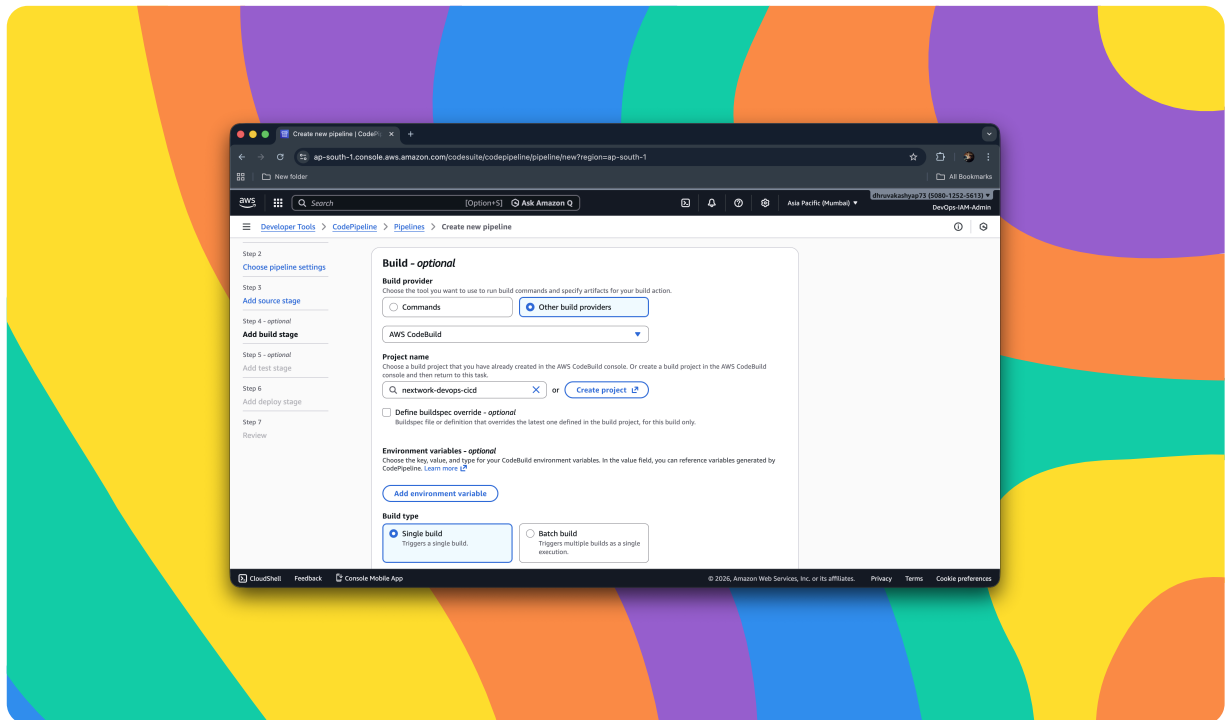
Webhook events are important because they automatically trigger the pipeline whenever new code is pushed to the selected GitHub branch. This removes the need to manually start builds and deployments, making the process truly continuous and faster. It also ensures every change is tested and deployed consistently with less chance of missing updates or causing human errors.





## Build Stage

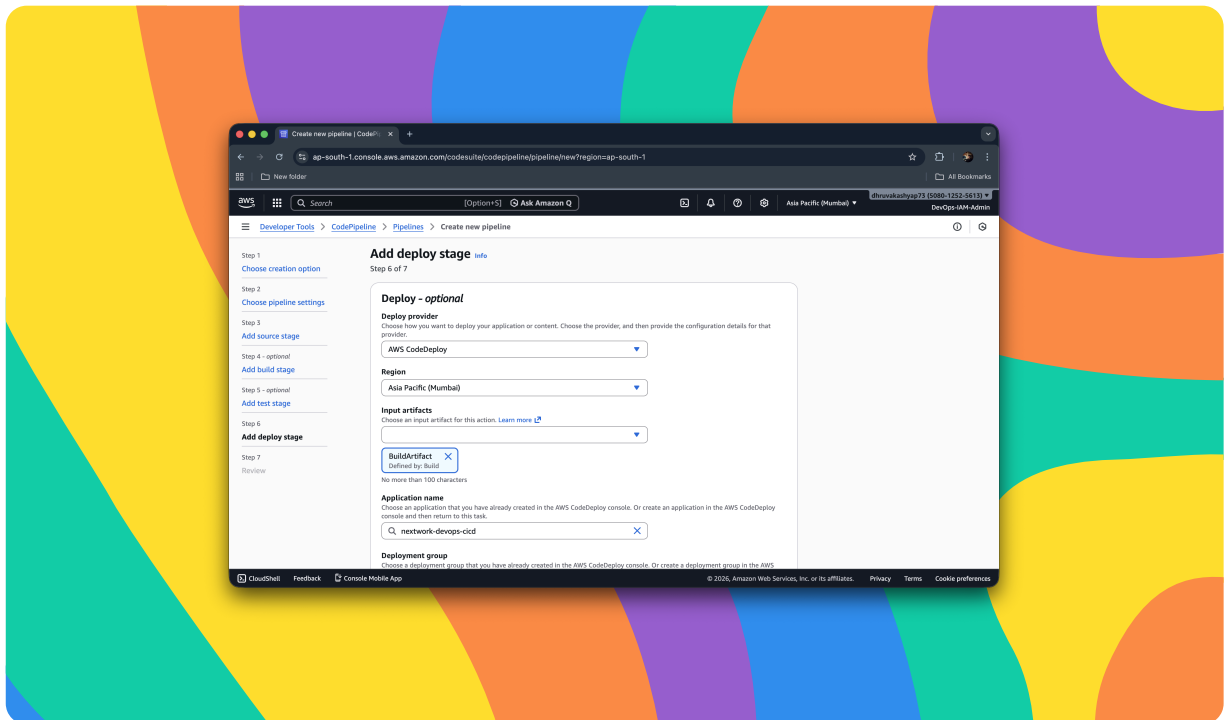
SourceArtifact is the input artifact for the Build stage because it contains the latest source code packaged by the Source stage from GitHub. CodeBuild needs this artifact so it can compile, test, and package the application into a deployable build output. This ensures the Build stage always works with the exact version of code that triggered the pipeline execution.





## Deploy Stage

In the Deploy stage, we configured CodePipeline to use AWS CodeDeploy to deploy the BuildArtifact produced by CodeBuild to our target EC2 instance. We selected the existing CodeDeploy application and deployment group so the pipeline knows exactly where and how to deploy the new build. We also enabled automatic rollback on stage failure to automatically restore the last working version if deployment fails.





## Success!

We updated the index.jsp file by adding a new line i.e, "<p>If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!</p>" inside the <body> section to display a confirmation message in the web app. After saving the change, we used git add, git commit, and git push origin master to push the update to GitHub. This triggers CodePipeline automatically through webhooks and starts a new build and deployment.

After pushing the code change to GitHub, CodePipeline automatically triggered a new pipeline execution through webhook events. The Source stage pulled the latest commit and displayed the commit message and ID, then CodeBuild built a new artifact and CodeDeploy deployed it to the EC2 instance. When all stages turned green, it confirmed the update was successfully built and deployed without manual steps.

We confirmed the CI/CD pipeline was working by pushing a code change to GitHub and seeing CodePipeline automatically start a new execution. After the Build and Deploy stages completed successfully (turned green), we opened the EC2 Public IPv4 DNS in a browser. When the new line appeared in the web app, it proved the update was deployed automatically through the pipeline.



**Dhruva Kashyap**

NextWork Student

[nextwork.org](https://nextwork.org)

## Hello NextWork!

This is my NextWork web application working!

If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :D

If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!

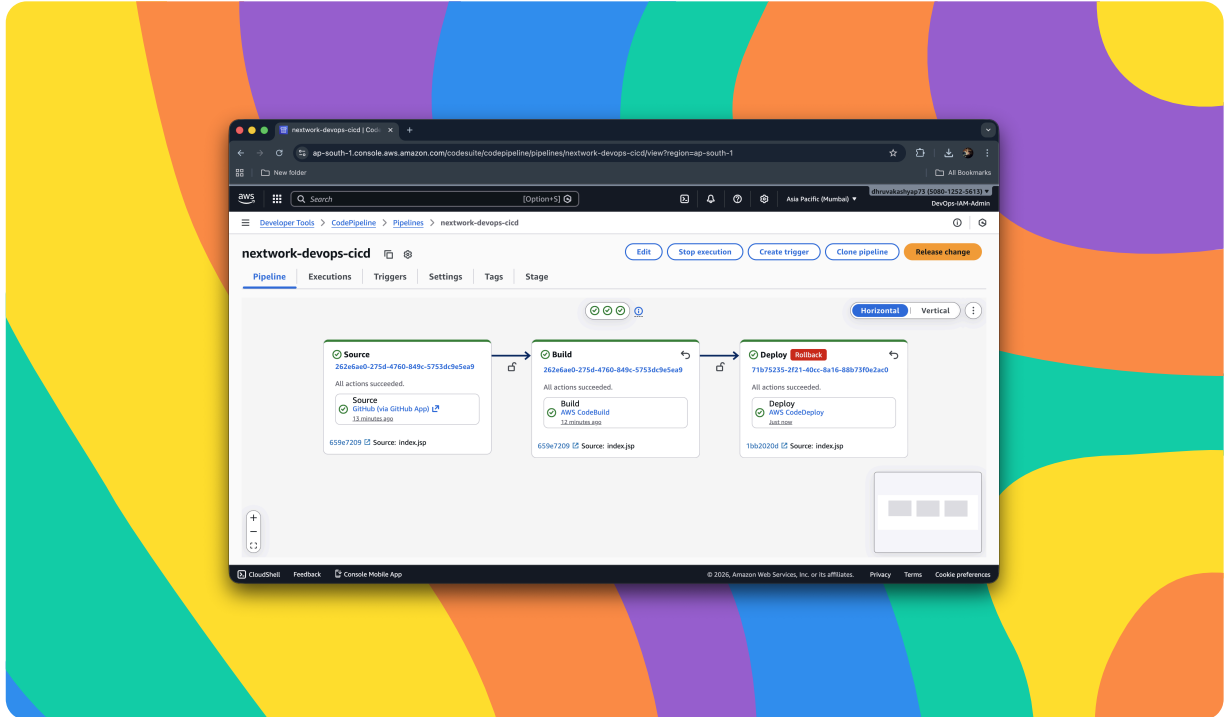


## Testing the Pipeline

We initiated the rollback on the Deploy stage of the CodePipeline workflow. This rollback only reverted the deployed application version on the EC2 instance to the last working release, without changing the Source or Build stages. It helped restore a stable version quickly in case the latest deployment caused issues.

The Source and Build stages are unaffected by this rollback because the rollback was initiated only on the Deploy stage to revert the application running on the EC2 instance. CodePipeline keeps using the latest commit for Source and the latest generated artifact for Build, but switches the deployed revision back to the previous successful one. This is useful when the code and build are fine, but the deployment result has issues and you need to quickly restore a stable version.

After the rollback, the web application reverted to the previous working version and the new `<p>` line that was added for testing was no longer visible. This showed that the Deploy stage successfully restored the last successful deployment on the EC2 instance. It confirmed that rollback worked correctly by removing the latest deployed change from the live site.





[nextwork.org](https://nextwork.org)

# The place to learn & showcase your skills

Check out [nextwork.org](https://nextwork.org) for more projects

